Assignment 1:
Exploring OpenGL Programming

NAME: XU MINHAO
STUDENT NUMBER: 2024533139

1 INTRODUCTION

This assignment implements a small interactive OpenGL application for rendering a three-dimensional mesh. The program creates an OpenGL 3.3 core-profile context, loads a sphere from an object file, and renders it with depth testing and per-fragment lighting. In addition to the basic rendering pipeline, the application supports first-person camera movement, mouse-look control, and keyboard rotation. The main purpose of the assignment is to become familiar with the relationship between CPU-side OpenGL resource management, shader programs, vertex attributes, and the transformation pipeline.

2 IMPLEMENTATION DETAILS

2.1 Window and OpenGL Initialization

The `Window` class wraps GLFW window creation. It requests an OpenGL 3.3 core context, makes the context current, and initializes the OpenGL function pointers through GLAD. The initial viewport is obtained from the framebuffer size, and a framebuffer-size callback updates the viewport when the window is resized. Vertical synchronization is enabled through `glfwSwapInterval(1)`.

2.2 Mesh Loading and Vertex Specification

The `Mesh` class reads the vertex positions, vertex normals, and triangle face records from the supplied `.object` file. Each face is expanded into three vertices, where every vertex stores a position and a normal. The data are uploaded to a vertex buffer object (VBO), while an element buffer object (EBO) stores the indices of the triangles. A vertex array object (VAO) records the vertex layout: attribute 0 contains three floating-point position values and attribute 1 contains three floating-point normal values. During drawing, the VAO is bound and `glDrawElements` renders the triangles.

2.3 Shader Pipeline

The vertex shader transforms each input position from model space to clip space:

$$\boldsymbol{p}_{clip} = PVM \begin{bmatrix} \boldsymbol{p}_{model} \\ 1 \end{bmatrix}.$$

It also transforms the normal with the normal matrix and passes the world space position and normal to the fragment shader. The fragment shader computes ambient, diffuse, and specular terms. In particular, the diffuse term is based on the non-negative dot product between the normalized normal and light direction, while the specular term uses the reflected light direction and a shininess exponent of 32:

$$I = k_a C + k_d \max(\boldsymbol{n} \cdot \boldsymbol{l}, 0) C + k_s \left(\max(\boldsymbol{v} \cdot \boldsymbol{r}, 0)\right)^{32}.$$

The object color is orange-brown, and the light is placed at $(2, 3, 2)$ in world space.

2.4 Camera and Interaction

The camera stores its position, viewing direction, yaw, and pitch. The view matrix is constructed from the camera's forward, right, and up vectors, which are obtained using vector cross products and normalization. A perspective projection with a 45-degree vertical field of view, near plane 0.1, and far plane 100.0 is recomputed using the current framebuffer aspect ratio.

The `W`, `A`, `S`, and `D` keys move the camera relative to its viewing direction, while `E` and `Q` move it along the world-up axis. Movement is multiplied by frame time, so it is independent of the frame rate. The left and right arrow keys change yaw, and the up and down arrow keys change pitch. Mouse motion provides a second way to control yaw and pitch; pitch is clamped to the interval $[-89°, 89°]$ to avoid the singularity when the camera looks directly along the up axis. Pressing Escape closes the window.

2.5 Multiple Objects and Motion

The scene is not limited to a single mesh. The program creates several `ScreenObject` entries, each storing a mesh pointer, a model matrix, a position, a velocity, a scale, a normal matrix, and an object color. These entries are placed in a `std::array<ScreenObject, 3>` and rendered in a single loop. This allows the same shader pipeline to be reused for multiple meshes, while still keeping each object's transform data independent.

At every frame, the model matrix is recomputed from the object's current position and scale, and the normal matrix is updated accordingly. The camera and light parameters are then uploaded to the shader as uniforms before the meshes are drawn. A simple animation is also included by updating each object's position with velocity multiplied by the elapsed time:

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \mathbf{v}\,\Delta t.$$

This makes the objects move smoothly across the scene while maintaining a consistent transformation pipeline for each rendered mesh.

3 RESULTS

When executed, the application opens a 1280×720 GLFW window and renders the sphere against a blue-gray background. The depth buffer is enabled, so nearer parts of the sphere correctly occlude farther parts. The lighting model produces a constant ambient contribution, a diffuse shading gradient across the surface, and a visible

© 2026
Computer Graphics I Course Assignment

specular highlight near the reflection direction. The rendered object remains correctly framed when the window is resized because both the viewport and the projection aspect ratio are updated.

The interactive camera can orbit and translate around the object using the mouse and keyboard controls described above. This also makes the effects of the view matrix and perspective projection directly observable. Overall, the program demonstrates the complete path from mesh-file data, through OpenGL buffers and shader stages, to a depth-tested, interactively viewed image.

4 BUILD DIAGNOSTICS AND FIX NOTE

During the compilation stage, the project initially failed in the object-update loop inside the main rendering routine. The compiler reported that the assignment and compound-assignment operators were unavailable for the loop variable, indicating that the object was treated as a constant copy rather than a mutable element of the array. In particular, the loop used `for (auto const object : objects)`, which prevents updates to `object.model`, `object.normalMatrix`, and `object.position`. The correct pattern is to iterate by reference, `for (auto &object : objects)`, so that the position, velocity, and model matrix can be recomputed each frame. This fix resolves the reported `operator=` and `operator+=` errors and allows the animation and per-frame matrix updates to proceed normally.

5 CONCLUSION

This assignment provided practical experience with the core components of an OpenGL rendering pipeline. The most important implementation details were preserving the VAO/VBO/EBO state relationships, passing data between shader stages with matching interfaces, and keeping the camera matrices and framebuffer aspect ratio consistent. The final program combines these components into a compact interactive renderer with basic Phong-style illumination.